DBMS PRACTICALS

Four SQL Programs — With Outputs

Included: (1) DDL/DML and constraints, (2) foreign key and referential integrity, (3) WHERE clauses and aggregate functions, (4) subqueries and JOINs.

Outputs are representative result tables for the sample data shown. Run programs in MySQL. Skip CREATE/INSERT statements if the same database or records already exist.

Practical 1: Database, Constraints, DDL and DML

Aim: Create a database and table with PRIMARY KEY, UNIQUE, CHECK and NOT NULL; perform INSERT, UPDATE and DELETE.

Create database/table and insert records

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Age INT CHECK (Age >= 18),
  City VARCHAR(50)
);

INSERT INTO Student (StudentID, Name, Email, Age, City) VALUES
(1, 'Rahul', 'rahul@gmail.com', 20, 'Ahmedabad'),
(2, 'Priya', 'priya@gmail.com', 22, 'Surat'),
(3, 'Amit', 'amit@gmail.com', 19, 'Vadodara');

SELECT * FROM Student;
```

StudentID	Name	Email	Age	City
1	Rahul	rahul@gmail.com	20	Ahmedabad
2	Priya	priya@gmail.com	22	Surat
3	Amit	amit@gmail.com	19	Vadodara

UPDATE and output

```
UPDATE Student SET City = 'Mumbai' WHERE StudentID = 1;
SELECT * FROM Student;
```

StudentID	Name	Email	Age	City
1	Rahul	rahul@gmail.com	20	Mumbai
2	Priya	priya@gmail.com	22	Surat
3	Amit	amit@gmail.com	19	Vadodara

DELETE and output

```
DELETE FROM Student WHERE StudentID = 3;
SELECT * FROM Student;
```

StudentID	Name	Email	Age	City
1	Rahul	rahul@gmail.com	20	Mumbai
2	Priya	priya@gmail.com	22	Surat

Constraint checks — these examples should fail

```
-- Duplicate primary key:
INSERT INTO Student VALUES (1, 'Same ID', 'sameid@gmail.com', 20, 'Pune');
-- Duplicate email:
INSERT INTO Student VALUES (4, 'Duplicate Email', 'rahul@gmail.com', 21, 'Pune');
-- NOT NULL violation:
INSERT INTO Student VALUES (5, NULL, 'nullname@gmail.com', 21, 'Pune');
-- CHECK violation:
INSERT INTO Student VALUES (6, 'Underage', 'young@gmail.com', 16, 'Pune');
```

Result: Database and table created; constraints and DML operations demonstrated.

Practical 2: Foreign Key and Referential Integrity

Aim: Create parent/child tables and enforce referential integrity.

```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Department (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(50) NOT NULL,
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

INSERT INTO Department VALUES
(101, 'Computer Science'),
(102, 'Information Technology'),
(103, 'Commerce');

INSERT INTO Student VALUES
(1, 'Rahul', 101), (2, 'Priya', 102), (3, 'Amit', 101);

SELECT * FROM Department;
SELECT * FROM Student;
```

Output: Department

DeptID	DeptName
101	Computer Science
102	Information Technology
103	Commerce

Output: Student

StudentID	StudentName	DeptID
1	Rahul	101
2	Priya	102
3	Amit	101

Join output

```
SELECT Student.StudentID, Student.StudentName, Department.DeptName
FROM Student INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

StudentID	StudentName	DeptName
1	Rahul	Computer Science
2	Priya	Information Technology
3	Amit	Computer Science

Test referential integrity

```
-- Expected to fail: DeptID 999 does not exist
INSERT INTO Student VALUES (4, 'Neha', 999);

-- Expected to fail while students reference department 101
DELETE FROM Department WHERE DeptID = 101;

-- Cascades the ID change to matching student rows
UPDATE Department SET DeptID = 105 WHERE DeptID = 102;
```

The invalid insert and restricted delete are rejected. ON UPDATE CASCADE updates related Student rows when a referenced department ID changes.

Result: Foreign key and referential integrity implemented.

Practical 3: WHERE Clauses and Aggregate Functions

Aim: Filter records using WHERE conditions and calculate aggregate results.

Sample rows used for outputs:

StudentID	StudentName	DeptID	Marks
1	Rahul	101	85
2	Priya	102	90
3	Amit	101	75
4	Neha	103	88
5	Karan	102	95

WHERE =

```
SELECT StudentName, DeptID FROM Student WHERE DeptID = 101;
```

StudentName	DeptID
Rahul	101
Amit	101

WHERE >

```
SELECT StudentID, StudentName FROM Student WHERE StudentID > 2;
```

StudentID	StudentName
3	Amit
4	Neha
5	Karan

WHERE <

```
SELECT StudentID, StudentName FROM Student WHERE StudentID < 4;
```

StudentID	StudentName
1	Rahul
2	Priya
3	Amit

WHERE AND

```
SELECT StudentName, DeptID FROM Student WHERE StudentID > 2 AND DeptID = 102;
```

StudentName	DeptID
Karan	102

WHERE OR

```
SELECT StudentName, DeptID FROM Student WHERE DeptID = 101 OR DeptID = 103;
```

StudentName	DeptID
Rahul	101
Amit	101
Neha	103

WHERE BETWEEN

```
SELECT StudentID, StudentName FROM Student WHERE StudentID BETWEEN 2 AND 4;
```

StudentID	StudentName
2	Priya
3	Amit
4	Neha

WHERE IN

```
SELECT StudentName, DeptID FROM Student WHERE DeptID IN (101, 102);
```

StudentName	DeptID
Rahul	101

StudentName	DeptID
Priya	102
Amit	101
Karan	102

WHERE NOT IN

```
SELECT StudentName, DeptID FROM Student WHERE DeptID NOT IN (101, 102);
```

StudentName	DeptID
Neha	103

WHERE LIKE

```
SELECT StudentName FROM Student WHERE StudentName LIKE 'R%';
```

StudentName
Rahul

WHERE IS NULL

```
SELECT StudentName FROM Student WHERE DeptID IS NULL;
```

Empty set — no rows have a NULL DeptID in this sample.

WHERE IS NOT NULL

```
SELECT StudentName FROM Student WHERE DeptID IS NOT NULL;
```

StudentName
Rahul
Priya
Amit
Neha
Karan

Aggregate functions

```
SELECT COUNT(*) AS TotalStudents FROM Student;
SELECT SUM(Marks) AS TotalMarks FROM Student;
SELECT AVG(Marks) AS AverageMarks FROM Student;
SELECT MAX(Marks) AS HighestMarks FROM Student;
SELECT MIN(Marks) AS LowestMarks FROM Student;
```

Function	Output
COUNT(*)	5
SUM(Marks)	433
AVG(Marks)	86.6
MAX(Marks)	95
MIN(Marks)	75

GROUP BY and HAVING

```
SELECT DeptID, COUNT(*) AS TotalStudents
FROM Student GROUP BY DeptID;

SELECT DeptID, AVG(Marks) AS AverageMarks
FROM Student GROUP BY DeptID
HAVING AVG(Marks) > 85;
```

DeptID	TotalStudents
101	2
102	2
103	1

DeptID	AverageMarks
102	92.5
103	88.0

Result: WHERE, aggregate functions, GROUP BY and HAVING demonstrated.

Practical 4: Subqueries and JOIN Operations

Aim: Retrieve records using subqueries and combine related rows using JOINs.

Sample Student data:

StudentID	StudentName	DeptID	Marks
1	Rahul	101	85
2	Priya	102	90
3	Amit	101	75
4	Neha	103	88
5	Karan	102	95

Sample Department data includes an unused department to demonstrate RIGHT JOIN:

DeptID	DeptName
101	Computer Science
102	Information Technology
103	Commerce
104	Physics

Subquery 1: Students scoring above average

```
SELECT StudentName, Marks FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

StudentName	Marks
Priya	90
Karan	95

Subquery 2: Students in Computer Science

```
SELECT StudentName FROM Student
WHERE DeptID = (
  SELECT DeptID FROM Department
  WHERE DeptName = 'Computer Science'
);
```

StudentName
Rahul
Amit

Subquery 3: Student with highest marks

```
SELECT StudentName, Marks FROM Student
WHERE Marks = (SELECT MAX(Marks) FROM Student);
```

StudentName	Marks
Karan	95

Subquery 4: Students scoring more than Rahul

```
SELECT StudentName, Marks FROM Student
WHERE Marks > (
  SELECT Marks FROM Student WHERE StudentName = 'Rahul'
);
```

StudentName	Marks
Priya	90
Neha	88
Karan	95

INNER JOIN

```
SELECT Student.StudentID, Student.StudentName, Department.DeptName
FROM Student INNER JOIN Department
ON Student.DeptID = Department.DeptID;
```

StudentID	StudentName	DeptName
1	Rahul	Computer Science
2	Priya	Information Technology

StudentID	StudentName	DeptName
3	Amit	Computer Science
4	Neha	Commerce
5	Karan	Information Technology

LEFT JOIN

```
SELECT Student.StudentName, Department.DeptName
FROM Student LEFT JOIN Department
ON Student.DeptID = Department.DeptID;
```

StudentName	DeptName
Rahul	Computer Science
Priya	Information Technology
Amit	Computer Science
Neha	Commerce
Karan	Information Technology

RIGHT JOIN

```
SELECT Student.StudentName, Department.DeptName
FROM Student RIGHT JOIN Department
ON Student.DeptID = Department.DeptID;
```

StudentName	DeptName
Rahul	Computer Science
Amit	Computer Science
Priya	Information Technology
Karan	Information Technology
Neha	Commerce
NULL	Physics

CROSS JOIN

```
SELECT Student.StudentName, Department.DeptName
FROM Student CROSS JOIN Department;
```

Output contains 5 × 4 = 20 rows. First few rows:

StudentName	DeptName
Rahul	Computer Science
Rahul	Information Technology
Rahul	Commerce
Rahul	Physics
Priya	Computer Science
...	...

Result: Subqueries and INNER, LEFT, RIGHT and CROSS JOIN operations demonstrated.